

The QR Algorithm, Factor Flipping, and Inverse Iteration

Biajid Ejaj Chowdhury

June 23, 2026

1 The Big Picture

Suppose we have a real symmetric matrix $A \in \mathbb{R}^{m \times m}$. Our goal is to understand how to compute its entire spectrum of eigenvalues and eigenvectors simultaneously. An eigenpair (λ, v) satisfies the fundamental relation:

$$Av = \lambda v, \quad v \neq 0.$$

While single eigenpairs can be isolated using standard vector methods, real commercial computing frameworks rely on the global convergence properties of the **QR Algorithm**. Developed independently in the late 1950s by John Francis and Vera Kublanovskaya, this approach computes the complete set of eigenpairs at once.

Note on Prerequisites: Before we proceed, we assume that the reader is already familiar with the standard power iteration method for finding a single dominant eigenvalue. We will use that exact intuition as our foundational building block here.

To find all the eigenpairs of a matrix simultaneously, our plan is:

1. Run the unshifted QR algorithm loop via repeated factorization and factor-flipping ($A_k = R_k Q_k$) to drive the matrix into a clean diagonal or upper-triangular form.
2. Use an automated convergence checker function to verify that our subdiagonal elements have decayed to zero.
3. Extract the pure eigenvalues directly from the diagonal, and pass them into a perturbed **Inverse Iteration** routine using random starting vectors to pull out the exact corresponding eigenvectors.

However, before running the loop, we need to understand the underlying geometry.

2 The Core Problem: Multi-Vector Power Method Collapse

To understand why the QR algorithm functions, we must examine what happens when we attempt to run the standard power method on multiple vectors at the same time. Let $X_0 \in \mathbb{R}^{m \times m}$ be a matrix whose columns are random, linearly independent starting vectors. If we simply multiply this matrix by A repeatedly, we generate a sequence of matrices:

$$X_k = A^k X_0.$$

Recall what A does to space geometrically. Every matrix has a dominant direction of stretching governed by its largest eigenvalue magnitude $|\lambda_1|$. When we multiply our random matrix by A over and over, every single column vector gets pulled and dragged toward that absolute strongest stretching axis.

As $k \rightarrow \infty$, every column in X_k tilts toward the exact same dominant eigenvector v_1 . Consequently, the columns become nearly identical and lose their linear independence. The entire multi-vector grid collapses numerically onto a single line, preventing us from isolating any of the remaining eigenvalues or eigenvectors.

3 The Fix: Simultaneous Iteration via QR Separation

To stop the columns from collapsing onto the same dominant vector, we must force them to remain separated. This is achieved by enforcing strict orthogonality after each multiplication step using a reduced QR factorization. This upgraded framework is called **Simultaneous Iteration**.

Let us choose the simplest set of orthogonal vectors possible as our initial guess: the columns of the Identity Matrix I . We set our global tracking frame $\underline{Q}_0 = I$. The loop proceeds for $k = 1, 2, \dots$ as follows:

1. Multiply the current orthogonal frame by A to advance the power step:

$$X_k = A \underline{Q}_{k-1}$$

2. Straighten the distorted vectors back out into a perfectly perpendicular grid using a QR decomposition:

$$\underline{Q}_k \underline{R}_k = X_k$$

This step-by-step orthogonalization acts like a cascading geometric restriction:

- The first column q_1 has a free pass. Because it is at the front, the QR step doesn't subtract anything from it. It gets multiplied by A over and over, meaning it is mathematically dragged until it snaps directly onto the first eigenvector v_1 .

- The second column q_2 also wants to tilt toward v_1 . But remember our QR rule: q_2 is strictly forced to stay perpendicular to q_1 . As q_1 locks onto v_1 , q_2 is physically forbidden from pointing in that direction. It has to settle for the next strongest stretching force available in the remaining space, which drags it until it locks onto the second eigenvector v_2 .
- This domino effect cascades down all the columns. Each vector occupies a unique, orthogonal channel corresponding to a specific strength of the matrix's stretching power.

As $k \rightarrow \infty$, the columns of the cumulative tracking matrix $\underline{Q}_k$ perfectly sort themselves out until they align exactly with the true eigenvectors.

4 The Rayleigh Quotient Perspective

Now that we have this rotating grid of tracking vectors $\underline{Q}_k$ slowly aligning with the true eigenvector basis, how do we actually find the eigenvalues? We use a tool called the ****Rayleigh Quotient similarity transformation****.

If we wrap our matrix A with our current tracking vectors, we look at our original matrix from the perspective of our tracking frame:

$$A_k = \underline{Q}_k^T A_0 \underline{Q}_k.$$

Think about what happens at the limit. Since $\underline{Q}_k \rightarrow V$ (the true eigenvector matrix), this transformation approaches $V^T A_0 V$. For a real symmetric matrix, this maps directly to a pure diagonal matrix D :

$$A_k \rightarrow V^T(A_0 V) = V^T(VD) = (V^T V)D = ID = D.$$

This is our window to the eigenvalues. When $\underline{Q}_k$ reaches the true eigenvectors, the off-diagonal elements are driven to zero, leaving the eigenvalues sitting directly on the main diagonal.

5 The Brilliant Shortcut: Flipping the Factors

Maintaining the historical product matrix $\underline{Q}_k = Q_1 Q_2 \dots Q_k$ is computationally bloated. The breakthrough of the QR algorithm is a mind-blowing algebraic shortcut that executes this entire multi-vector power projection implicitly inside the matrix itself.

The Algebraic Derivation

Let us define our starting matrix as $A_0 = A$. In our loop, we simply take the local QR decomposition of our current matrix and immediately multiply the factors in reverse order:

$$A_{k-1} = Q_k R_k \implies A_k = R_k Q_k.$$

Let us watch this unfold step-by-step for the first two iterations to see how this simple "flip step" automatically runs the multi-vector power method behind the scenes.

Iteration 1

We compute the local decomposition:

$$A_0 = Q_1 R_1.$$

Isolating R_1 by multiplying by Q_1^T from the left yields:

$$R_1 = Q_1^T A_0.$$

Now look at what happens when we compute our favorite flip step ($A_1 = R_1 Q_1$) by substituting that R_1 expression right into it:

$$A_1 = (Q_1^T A_0) Q_1 = Q_1^T A_0 Q_1.$$

Look at that final expression! The single local flip has automatically generated the first Rayleigh quotient.

Iteration 2

The loop repeats. We take the QR split of our new matrix A_1 :

$$A_1 = Q_2 R_2 \implies R_2 = Q_2^T A_1. \quad \text{Then we flip: } A_2 = R_2 Q_2.$$

Let us substitute our expression for R_2 right into the flip step:

$$A_2 = (Q_2^T A_1) Q_2.$$

Now, unpack A_1 by substituting our result from the first iteration ($A_1 = Q_1^T A_0 Q_1$) right into the middle:

$$A_2 = Q_2^T (Q_1^T A_0 Q_1) Q_2.$$

Grouping the neighbor terms at the front and back using the associative property reveals:

$$A_2 = (Q_1 Q_2)^T A_0 (Q_1 Q_2).$$

The product $(Q_1 Q_2)$ is a single combined orthogonal matrix representing our cumulative tracking frame after two steps!

The General Connection

If we run this local flip for k steps, the algebra chains out exactly the same way every single time:

$$A_k = (Q_1 Q_2 \dots Q_k)^T A_0 (Q_1 Q_2 \dots Q_k) = \underline{Q}_k^T A_0 \underline{Q}_k.$$

This reveals the complete bridge:

- The right side accumulates a pure forward power iteration chain: $(\dots((A_0 Q_1) Q_2) Q_3 \dots)$. The matrix A_0 is continually operating on its own previous orthogonal operators, dragging them straight into the true eigenvector directions.
- The left side naturally piles up the corresponding transposes out front: $(\dots Q_3^T (Q_2^T (Q_1^T \dots)))$. These act as the inverse operation, stripping away the spatial cross-talk so the matrix updates itself into its own eigenvalues.

6 Extracting Eigenvectors via Inverse Iteration

Once the QR loop converges and the eigenvalues λ_i are exposed on the diagonal of A_k , we can isolate the individual eigenvectors using **Inverse Iteration**.

To find the eigenvector v_i corresponding to λ_i , we set up a shifted, nearly-singular linear system:

$$(A_0 - (\lambda_i + \epsilon)I)y = q_{guess},$$

where $\epsilon = 10^{-12}$ is a tiny numerical perturbation added to prevent absolute division-by-zero errors during matrix inversion.

The Pitfall of Static Initial Guesses

If we choose a static initial guess vector, such as $q_{guess} = \mathbf{1} = [1, 1, \dots, 1]^T$, the algorithm can completely fail. If the true eigenvector v_i happens to be perfectly orthogonal to the vector of ones, its component within that guess is exactly zero:

$$v_i \cdot \mathbf{1} = 0.$$

Multiplying a zero component by an inverse scaling factor still leaves it at zero, meaning the system has a blind spot and can never isolate that eigenvector.

The Robust Fix

To guarantee a non-zero projection across all possible eigenvector directions, we must choose a randomly generated vector `rand(m, 1)` as our starting point. Running two quick steps of this inverse power shift flushes out numerical noise and guarantees clean convergence to the true normalized eigenvector.

7 Convergence Verification : Checking Upper Triangular Form

Before looking at the final driver code, we need an automated way to monitor when the loop has finished its job. The function `check_ut` acts as our convergence criterion by inspecting the lower-triangular portion of the matrix at each step.

For an $m \times m$ matrix, the function scans down column i starting precisely at row $i + 1$ all the way to the final row m . If the absolute value of any subdiagonal element exceeds our hard numerical tolerance of 10^{-9} , convergence is not yet achieved, and the function short-circuits to return false.

MATLAB Implementation

```
1 function out = check_ut(A)
2     % Convergence checker for upper-triangular subdiagonal decay
3     % Input:  A    = The current transformed matrix A_k
4     % Output: out = Logical true if subdiagonal elements are within tolerance,
5
6     [m, ~] = size(A);
7     out = true;
8
9     % Scan columns from 1 up to m - 1
10    for i = 1 : m - 1
11        % Look at all elements below the diagonal entry A(i,i)
12        if any(abs(A(i + 1 : end, i)) > 1e-9)
13            out = false;
14            break;
15        end
16    end
17 end
```

Key Interpretation

Let us read this code slowly:

- **Vectorized Subdiagonal Scans:** Instead of nested element-by-element loops, the code slices columns vertically using `A(i + 1 : end, i)`. This isolates all subdiagonal positions for that column in a single pass.
- **Numerical Tolerance vs. Strict Zero:** In practical floating-point arithmetic, subdiagonal elements rarely become exactly 0. Setting a tolerance threshold ($\epsilon = 10^{-9}$) ensures the algorithm terminates gracefully when the values have decayed far enough.

- **Early Break Optimization:** The moment a single element fails the threshold, the function exits immediately via the `break` statement, protecting our computational budget from unnecessary work.

8 Complete MATLAB Implementation and Driver

We now organize everything into a clean, reusable eigensolver function. It integrates our unshifted QR factor-flipping loop, our convergence checker, and our random-start inverse iteration routine.

1. The Main Eigensolver Routine

```

1 function [vect, out] = qr_method(A)
2     % Complete QR Eigensolver Package
3     % Input:  A      = Real symmetric matrix
4     % Outputs: vect = Matrix containing normalized eigenvectors as columns
5     %          out  = Diagonal matrix containing sorted eigenvalues
6
7     AO = A;
8     [Q, R] = qr(A);
9     A1 = R * Q;
10    n = 1;
11    [m, ~] = size(A);
12
13    % Step 1: Run the QR factor-flipping loop
14    while ~check_ut(A1)
15        A = A1;
16        [Q, R] = qr(A);
17        A1 = R * Q;
18        n = n + 1;
19
20        if n > 100 * m
21            error('Method failed to converge within iteration limits!');
22        end
23    end
24
25    % Step 2: Extract and sort eigenvalues
26    raw_ev = diag(A1);
27    [sorted_ev, ~] = sort(raw_ev, 'descend');
28    vect = zeros(m, m);
29
30    % Step 3: Run Inverse Iteration with random starts for eigenvectors
31    for i = 1 : m
32        % Initialize random starting vector to avoid orthogonal blindspots
33        q = rand(m, 1);

```

```

34
35     % Formulate the perturbed shifted matrix
36     M = A0 - (sorted_ev(i) + 1e-12) * eye(m);
37
38     % Run two rapid steps of inverse iteration for robust convergence
39     q = M \ q;
40     q = q / norm(q);
41
42     q = M \ q;
43     vect(:, i) = q / norm(q);
44 end
45
46 % Format the final eigenvalue output into a diagonal matrix
47 out = diag(sorted_ev);
48 end

```

2. Driver Program (How to Use It)

Now the user only needs to pass a real symmetric matrix to test the implementation.

```

1  clear; clc;
2
3  % Create a test symmetric matrix
4  A = [4 1 2;
5       1 5 3;
6       2 3 6];
7
8  % Call our complete solver
9  [eigenvectors, eigenvalues] = qr_method(A);
10
11 disp('Computed Eigenvalue Diagonal Matrix:');
12 disp(eigenvalues);
13
14 disp('Computed Eigenvector Column Matrix:');
15 disp(eigenvectors);

```

9 Key Points

1. Plain multi-vector power iteration fails because every column vector collapses onto the single dominant eigenvector axis.
2. Simultaneous iteration fixes this by executing a QR factorization at each step, forcing the tracking vectors to stay perfectly perpendicular.

3. The QR loop shortcut ($\mathbf{A} = \mathbf{R} * \mathbf{Q}$) implicitly computes the global Rayleigh transformation without tracking historical matrix frames.
4. The right-hand factor product drives the system forward into the eigen-directions, while the left-hand transpose components undo the spatial rotation.
5. Inverse iteration requires a randomized initial guess vector (`rand(m,1)`) because a static guess can land in a mathematical blind spot orthogonal to the true eigenvector.

References

- Laurene V. Fausett, *Applied Numerical Analysis Using MATLAB*, 2nd Edition. Prentice Hall, 2008.
- Radhey S. Gupta, *Elements of Numerical Analysis*, 2nd Edition. Cambridge University Press, 2015.
- Steven J. Leon, *Linear Algebra with Applications*, 9th Edition. Pearson, 2014.
- Chris Rycroft, *Harvard AM205 Video 5.6 - QR algorithm*. Available at: <https://youtu.be/TtPU4K0aQ9A>